DBT PORTFOLIO PROJECT

LAYER: INTERMEDIATE

DBT Intermediate Model

Linear Regression | OLS | Window Functions | Statistics | Snowflake

Project Summary

Implements Ordinary Least Squares (OLS) simple linear regression entirely in SQL -- using row number (time index) as the X predictor and the claim metric as Y. Derives beta_0 (intercept) and beta_1 (slope) analytically using the closed-form OLS formula, then outputs fitted values and residuals for every entity/date row.

Thomas Foster

Senior Data Analyst | Snowflake | DBT | Healthcare Analytics

github.com/thomasfoster | thomasfoster79@yahoo.com

1 PROJECT OVERVIEW

Model Name:	int_simple_linear_regression
Layer:	Intermediate
Materialization:	table
Source:	{{ source('healthcare', 'claims') }}
Key Outputs:	beta_0 (intercept), beta_1 (slope), fitted_value, residual
Audience:	Mart layer, Finance analysts, Actuarial teams

2 BUSINESS PROBLEM

Finance and actuarial teams needed to quantify linear trends in claims metrics per entity -- without exporting to Python or R. Is a payer's claim volume growing or shrinking over time? How fast? What would we expect next period? This model answers all three questions natively in SQL, producing fitted values Tableau can plot alongside actuals and residuals for anomaly detection.

- Trend direction: beta_1 > 0 = growing, < 0 = declining claim metric.
- Fitted values: Tableau plots regression line over actual time series.
- Residuals: Large residuals flag anomalous periods for investigation.
- Pure SQL: No Python, no external ML -- runs on Snowflake compute only.

3 TECHNICAL EXPLANATION

- base CTE: Assigns x = ROW_NUMBER() per entity ordered by period_date -- time becomes a numeric predictor.
- params CTE: Applies the closed-form OLS formula to compute beta_1 = (n*SUM(xy) - SUM(x)*SUM(y)) / (n*SUM(x^2) - SUM(x)^2).
- beta_0 computed as y_bar - beta_1 * x_bar (intercept from mean-centering identity).
- NULLIF() guards against division by zero when all x values are identical (single-row entities).
- Final SELECT: Joins base to params; computes fitted_value = beta_0 + beta_1 * x and residual = y - fitted_value.
- Materialized as TABLE for mart join performance.

Key Techniques: ROW_NUMBER() as time index, analytic OLS formula, NULLIF division guard, joined params CTE

4 FULL MODEL CODE

```
{{ config(materialized='table') }}

with base as (
    select
        entity_id,
        period_date,
        y_value as y,
        row_number() over (
            partition by entity_id order by period_date
        ) as x
    from {{ source('healthcare', 'claims') }}
),

params as (
    select
        entity_id,
        (sum(x * y) - count(*) * avg(x) * avg(y))
            / nullif(sum(x*x) - count(*) * avg(x) * avg(x), 0) as beta_1,
        avg(y) - (
            (sum(x * y) - count(*) * avg(x) * avg(y))
            / nullif(sum(x*x) - count(*) * avg(x) * avg(x), 0)
        ) * avg(x)                                              as beta_0
    from base
    group by entity_id
)

select
    b.entity_id,
    b.period_date,
    b.y                          as actual_value,
    p.beta_0,
    p.beta_1,
    p.beta_0 + p.beta_1 * b.x    as fitted_value,
    b.y - (p.beta_0 + p.beta_1 * b.x) as residual
from base b
join params p using (entity_id)
```

5 EXAMPLE OUTPUT

entity_id	period_date	actual_value	beta_0	beta_1	fitted_value	residual
E-001	2025-01-01	4250.75	4100.22	18.45	4118.67	132.08
E-001	2025-02-01	4380.20	4100.22	18.45	4137.12	243.08
E-002	2025-01-01	3180.10	3050.80	12.30	3063.10	117.00